INFORMATIK LK · WELT 1 · DS 8–12

Die Heldenschmiede



Klassen, Objekte, Vererbung

L1.1 ····· L1.2 ····· L1.3 ····· L1.4 ····· BOSS

Welt 1 im Überblick



DS 8

Level 1.1

Held

Attribute, Konstruktor, Getter, Schaden und Heilen

IDE + Lab 1.1



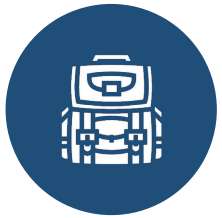
DS 9

Level 1.2

Monster & Kampf

Überladen, Monster, angreifen, Referenzen

IDE + Lab 1.2



DS 10

Level 1.3

Inventar

Gegenstand, Reihung von Objekten

IDE + Lab 1.3



DS 11

Level 1.4

Vererbung

Waffe, Trank, Schlüssel, Überschreiben

IDE + Lab 1.4



DS 12

Boss 1

Die Gildenhalle

Abi auf Zeit: ≈ 24 BE in 60 min

Papier + Online-Boss

Klasse vs. Objekt

KLASSE = Bauplan

Held
- name: Zeichenkette - leben: Ganzzahl - staerke: Ganzzahl …
+ c Held(name: Zeichenkette) + nimmSchaden(schaden: Ganzzahl): nichts …

`new Held("Aria")`

`new Held("Brann", 80, 14)`

OBJEKTE = Exemplare

aria: Held

name = "Aria"
leben = 100
staerke = 10
…

brann: Held

name = "Brann"
leben = 80
staerke = 14
…

Merke: Eine Klasse – beliebig viele Objekte. Jedes Objekt hat eigene Attributwerte.

Bestandteile einer Klasse

```
public class Held {
    private String name;
    private int leben;
    private int maxLeben;

    public Held(String name) {
        this.name = name;
        maxLeben = 100;
        leben = 100;
        // … staerke, gold, inventar
    }
    public int getLeben() { return leben; }
    public void heilen(int punkte) { … }
}
```

Klassenkopf
class + Klassenname

Attribute
private → Datenkapselung

Konstruktor
heißt wie die Klasse, kein Rückgabetyp, setzt Startwerte

Methoden
public – sondierend (getLeben) oder verändernd (heilen)

… = *hier gekürzt*

Die Klassenkarte im Abi-Format

Held
- name: Zeichenkette - leben: Ganzzahl - maxLeben: Ganzzahl - staerke: Ganzzahl - gold: Ganzzahl - inventar: Reihung von Gegenstand - anzahl: Ganzzahl - waffe: Waffe
+ c Held(name: Zeichenkette) + c Held(name: Zeichenkette, maxLeben: Ganzzahl, staerke: Ganzzahl) + getLeben(): Ganzzahl + nimmSchaden(schaden: Ganzzahl): nichts + aufnehmen(g: Gegenstand): Wahrheitswert + wertvollsterGegenstand(): Gegenstand …

Java	Klassenkarte
`int`	Ganzzahl
`double`	Kommazahl
`boolean`	Wahrheitswert
`char`	Zeichen
`String`	Zeichenkette
`void`	nichts
`Gegenstand[]`	Reihung von Gegenstand

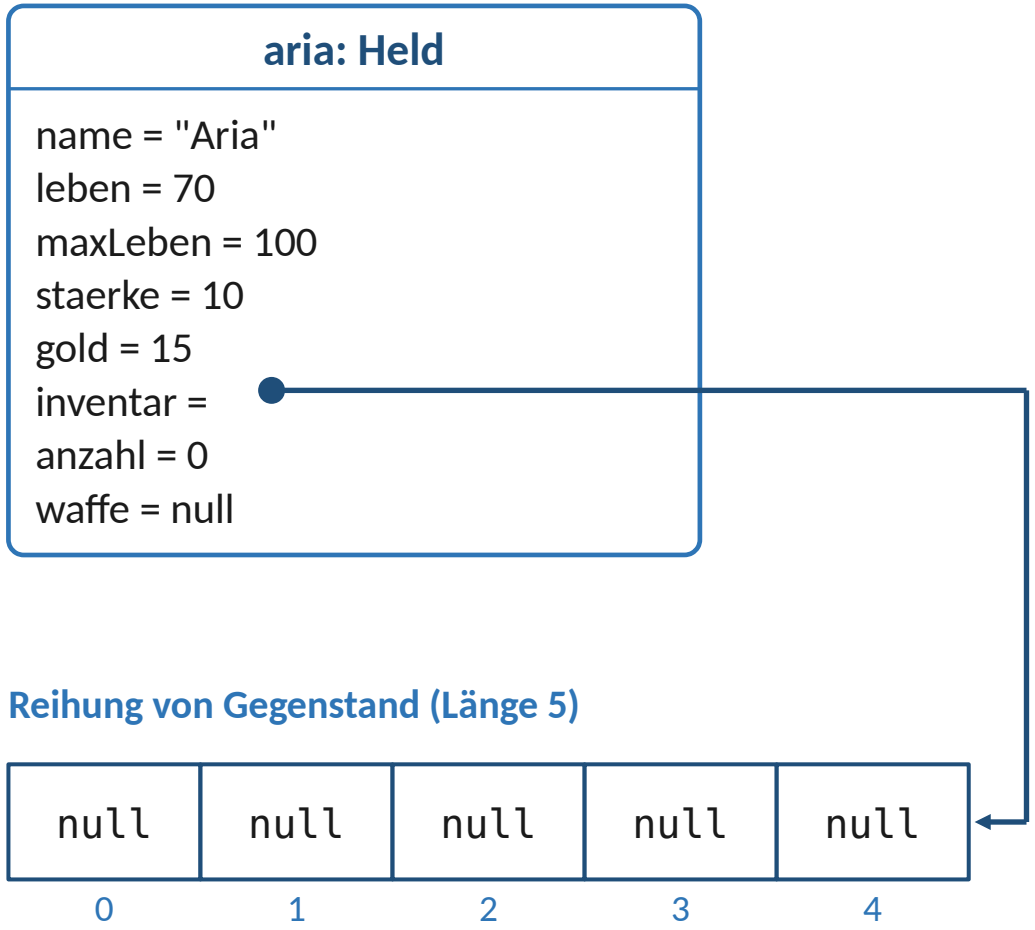
`+ / -`	Sichtbarkeit ist Pflicht: + public, - private
`c`	kennzeichnet Konstruktoren
`…`	nicht gezeigte Elemente
`: Typ`	Typ hinter dem Doppelpunkt (deutsch)

Die Objektkarte: ein Objekt zur Laufzeit

```
Held aria = new Held("Aria");
aria.nimmSchaden(30);
aria.erhalteGold(15);
```

So zeichnest du Objektkarten

- Kopf: objektname: Klasse
- aktuelle Werte statt Typen
- Zeichenketten in Anführungszeichen
- Referenz = Pfeil auf ein Objekt
- keine Referenz: null



Konstruktor, Überladen und this

```java
public Held(String name) {
    this.name = name;
    maxLeben = 100;
    leben = 100;
    staerke = 10;
    // … gold, inventar, anzahl, waffe
}
public Held(String name, int maxLeben, int staerke) {
    this.name = name;
    this.maxLeben = maxLeben;
    leben = maxLeben;
    this.staerke = staerke;
    // … gold, inventar, anzahl, waffe
}
```

Achtung: Auch im zweiten Konstruktor das Inventar anlegen!

Konstruktor

heißt wie die Klasse, hat keinen Rückgabetyp, wird mit new aufgerufen.

Überladen

gleicher Name, andere Parameterliste:

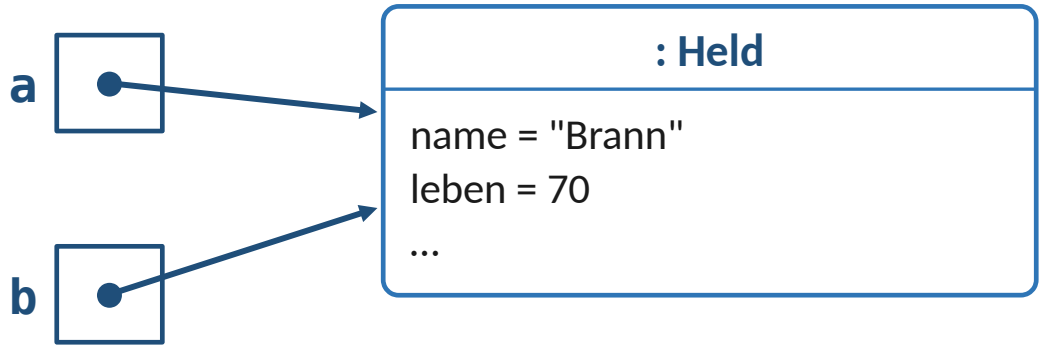
```
new Held("Aria")
new Held("Brann", 80, 14)
```

this

`this.name` = Attribut
`name` = Parameter

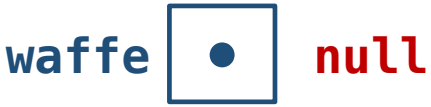
Referenzen und null

```
Held a = new Held("Brann");
Held b = a;
b.nimmSchaden(30);
println(a.getLeben());   // ?
```



Ausgabe: 70 – `Held b = a;` kopiert nur die Referenz. a und b zeigen auf dasselbe Objekt.

```
// in Held: waffe ist anfangs null
if (waffe == null) {
    return staerke;
}
return staerke + waffe.getSchaden();
```

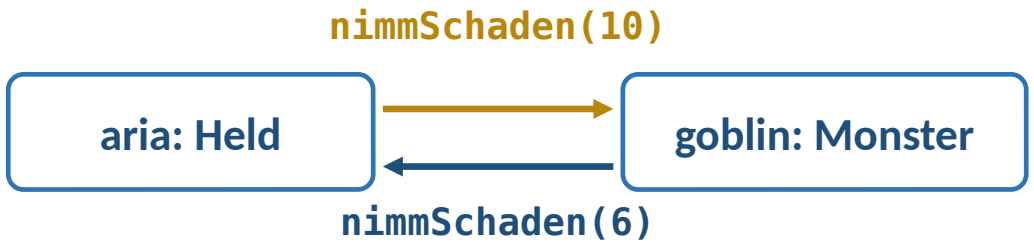


Kein Objekt – kein Pfeil. Ein Methodenaufruf auf null bricht das Programm ab. Darum: erst prüfen, dann zugreifen.

Monster und Kampf: Objekte schicken Botschaften

Monster
- name: Zeichenkette - leben: Ganzzahl - staerke: Ganzzahl
+ c Monster(name: Zeichenkette, leben: Ganzzahl, staerke: Ganzzahl) + getName(): Zeichenkette + getLeben(): Ganzzahl + getStaerke(): Ganzzahl + nimmSchaden(schaden: Ganzzahl): nichts + istBesiegt(): Wahrheitswert + angreifen(h: Held): nichts

```java
// in Held
public void angreifen(Monster m) {
    m.nimmSchaden(angriffswert());
}
// in Monster
public void angreifen(Held h) {
    h.nimmSchaden(staerke);
}
```

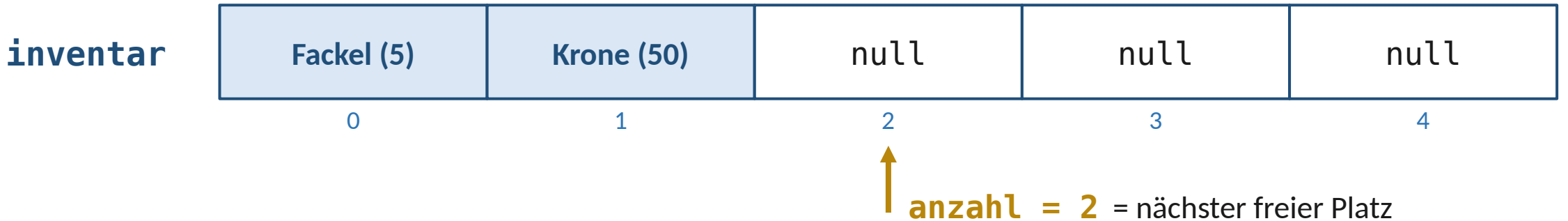


Nicht möglich: `m.leben = m.leben - 10;` – leben ist private. Objekte ändern einander nur über Methoden.

Assoziation: Der Held trägt ein Inventar



Held hat ein Attribut **`inventar: Reihung von Gegenstand`** → Assoziation: einfache Linie.

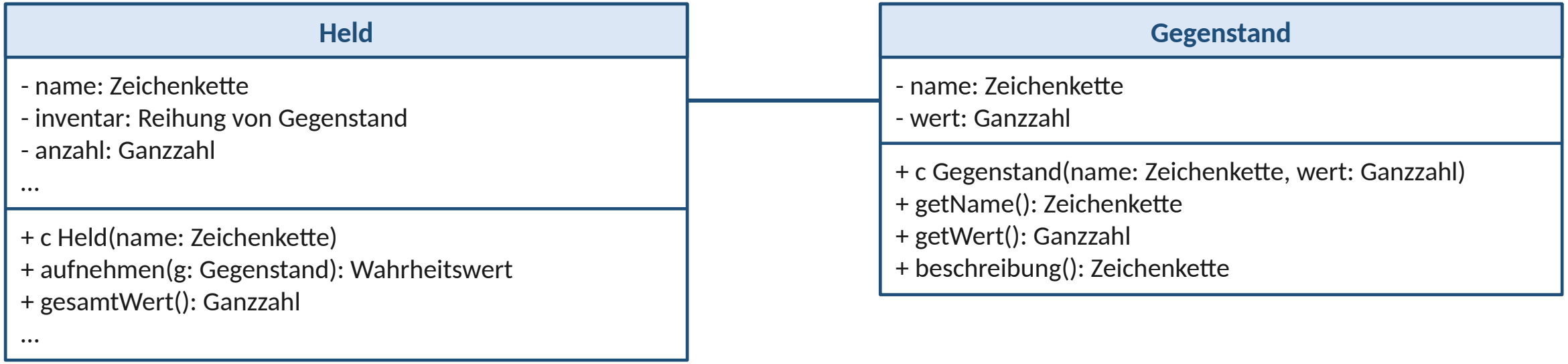



```
public boolean aufnehmen(Gegenstand g) {
    if (g == null || anzahl >= inventar.length) {
        return false;
    }
    inventar[anzahl] = g;
    anzahl++;
    return true;
}
```

1. erst prüfen: null? voll?
2. ans Ende: inventar[anzahl] = g
3. anzahl++ und true

Schleifen nur bis anzahl − 1
dahinter steht null

Klassendiagramm: Beziehungen zeichnen

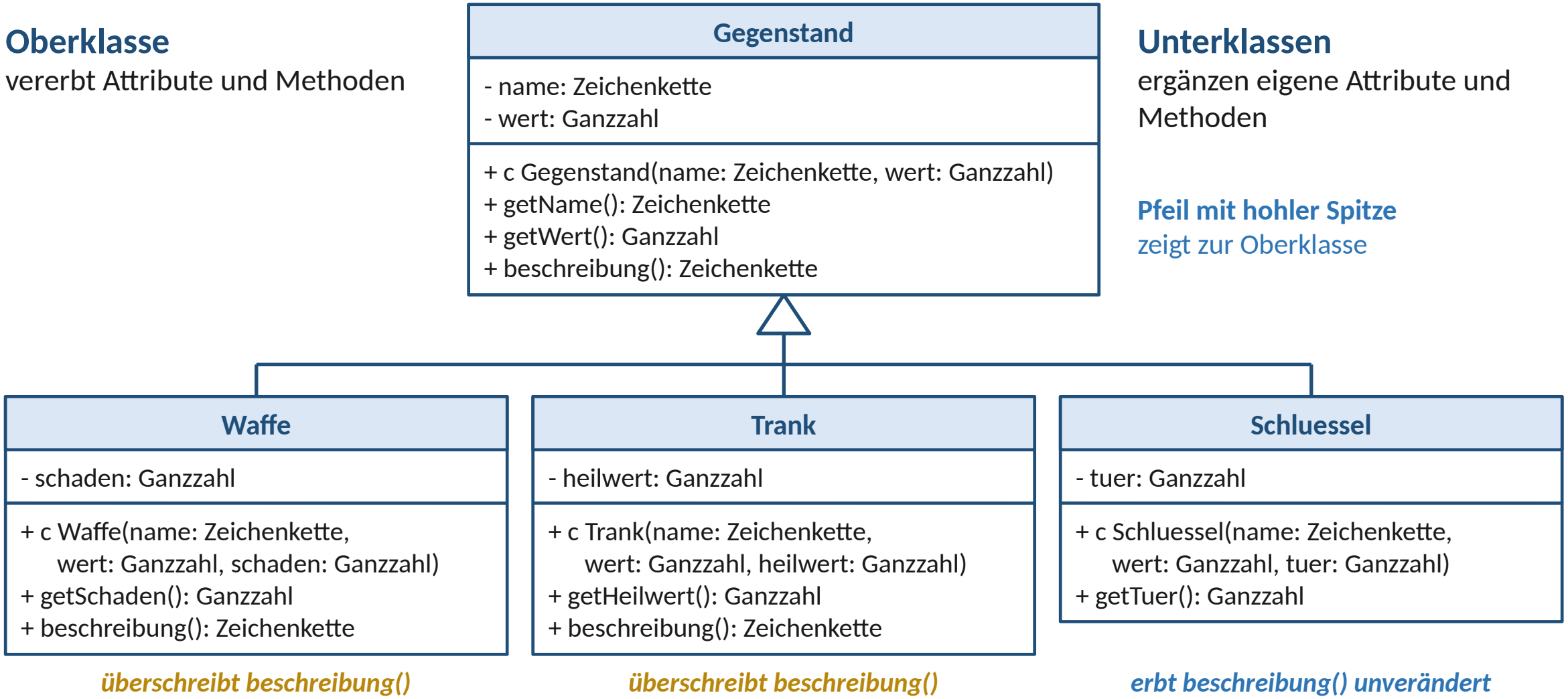


- einfache Linie zwischen den Klassen
- ohne Beschriftung, ohne Zahlen
- wenn ein Attribut den Typ der anderen Klasse hat

- keine Kardinalitäten wie 0..5 oder *
- keine Rauten, keine Rollennamen
- keine Pfeilspitzen an Assoziationen

Vererbung: Eine Waffe ist ein Gegenstand

Oberklasse
vererbt Attribute und Methoden



Unterklassen
ergänzen eigene Attribute und Methoden

Pfeil mit hohler Spitze
zeigt zur Oberklasse

überschreibt beschreibung()

überschreibt beschreibung()

erbt beschreibung() unverändert

super(...) und super.methode()

```
public class Waffe extends Gegenstand {
    private int schaden;

    public Waffe(String name, int wert, int schaden) {
        super(name, wert);
        this.schaden = schaden;
    }

    public String beschreibung() {
        return "Waffe: " + super.beschreibung()
               + ", Schaden " + schaden;
    }
}
```

`super(name, wert)`
ruft den Konstruktor der Oberklasse auf – muss die erste Anweisung sein.

`super.beschreibung()`
ruft die geerbte Version auf. Nötig, weil name und wert in Gegenstand private sind.

```
new Waffe("Schwert", 50, 12).beschreibung() → "Waffe: Schwert (50 Gold), Schaden 12"
```

Überschreiben: gleiche Signatur, neues Verhalten

Objekt	Version	Ergebnis
`new Gegenstand("Fackel", 5)`	**Gegenstand**	`"Fackel (5 Gold)"`
`new Waffe("Schwert", 50, 12)`	**Waffe (neu)**	`"Waffe: Schwert (50 Gold), Schaden 12"`
`new Trank("Heiltrank", 20, 30)`	**Trank (neu)**	`"Trank: Heiltrank (20 Gold), heilt 30"`
`new Schluessel("Rostiger Schlüssel", 2, 7)`	**Gegenstand (geerbt)**	`"Rostiger Schlüssel (2 Gold)"`

Überschreiben

Unterklasse definiert eine geerbte Methode mit gleicher Signatur neu.
Beispiel: beschreibung() in Waffe und Trank

Überladen

gleiche Klasse, gleicher Name, andere Parameterliste.
Beispiel: die zwei Konstruktoren von Held

Welche Methode läuft?

```
Gegenstand[] r = new Gegenstand[3];
r[0] = new Waffe("Axt", 30, 9);
r[1] = new Schluessel("Goldschlüssel", 40, 3);
r[2] = new Trank("Elixier", 25, 50);
for (Gegenstand g : r) {
    println(g.beschreibung());
}
```

? **Was gibt das Programm aus?**
Erst auf Papier, ohne Gerät.

2 min · im Team

1

2

3

Auflösung: Das Objekt entscheidet

r[0] ist eine Waffe	→ eigene Version	`"Waffe: Axt (30 Gold), Schaden 9"`
r[1] ist ein Schluessel	→ geerbte Version	`"Goldschlüssel (40 Gold)"`
r[2] ist ein Trank	→ eigene Version	`"Trank: Elixier (25 Gold), heilt 50"`

Typ der Variablen
bestimmt, welche Methoden aufgerufen werden dürfen (hier: alles aus Gegenstand).

Typ des Objekts
bestimmt zur Laufzeit, welche Version ausgeführt wird (dynamisches Binden).

Zuweisungskompatibilität

Regel: Ein Unterklassen-Objekt darf überall stehen, wo die Oberklasse erwartet wird – nicht umgekehrt.

✓	`Gegenstand g = new Waffe("Axt", 30, 9);`	Eine Waffe ist ein Gegenstand.
✗	`Waffe w = new Gegenstand("Fackel", 5);`	Nicht jeder Gegenstand ist eine Waffe.
✓	`g.beschreibung();`	kennt Gegenstand – es läuft die Waffe-Version.
✗	`g.getSchaden();`	getSchaden() gibt es in Gegenstand nicht.
✓	`held.aufnehmen(new Trank("Elixier", 25, 50));`	Parameter Gegenstand: jede Unterklasse passt.
✗	`held.ausruesten(g);`	erwartet Waffe – g hat den Typ Gegenstand.

Abi-Format: „Erläutern Sie die Klasse …“

AUFGABE (Beispiel)

Erläutern Sie die Klasse Held. Gehen Sie dabei auf die Attribute, die Konstruktoren und die Sichtbarkeiten ein.

(5 BE ≈ 12 min)

erläutern: einen Sachverhalt mit zusätzlichen Informationen und Beispielen verständlich machen

MUSTERANTWORT

Die Klasse Held beschreibt eine Spielfigur. Ihre Attribute, z. B. name, leben, maxLeben, staerke und gold, sind private: Sie können nur über öffentliche Methoden gelesen oder verändert werden (Datenkapselung). So stellt nimmSchaden sicher, dass leben nie unter 0 sinkt. Das Attribut inventar ist eine Reihung von Gegenstand mit fünf Plätzen, anzahl zählt die belegten Plätze. Es gibt zwei überladene Konstruktoren: Held(name) setzt Standardwerte (100 Leben, Stärke 10), Held(name, maxLeben, staerke) übernimmt die Werte aus den Parametern. Beide legen ein leeres Inventar an und setzen waffe auf null.

Kernpunkte: Das will der Erwartungshorizont sehen

- Zweck der Klasse in einem Satz
- Attribute nennen, private begründen (Datenkapselung)
- Konstruktoren: Anzahl, Parameter, was gesetzt wird
- Methoden als öffentliche Schnittstelle – mit Beispiel
- Beziehungen: Assoziation (inventar, waffe), ggf. Vererbung
- Fachbegriffe: Attribut, Konstruktor, Parameter, Rückgabewert

Typischer Fehler: den Code Zeile für Zeile vorlesen, statt zu erklären, wozu etwas da ist.

Abi-Check: Vor dem Abgeben



1 Signatur & Rückgabe

Signatur wie gefordert, return in jedem Pfad

Welt 1: aufnehmen liefert immer true oder false



2 Erst prüfen

vor jedem Zugriff: Leer-, Index-, Blatt-Abfrage

Welt 1: anzahl == 0? waffe == null?



3 Schleifengrenzen

Index von 0 bis length − 1

Welt 1: im Inventar: bis anzahl − 1



4 Struktur unverändert

falls gefordert: am Ende alles wie vorher

Welt 1: z. B. Inventar nicht umsortieren



5 Zeichenketten

nur erlaubte Operationen, Vergleich mit equals

Welt 1: getName().equals(name)



6 Kommentieren

unsichere Stellen kurz kommentieren

Welt 1: Annahmen offen hinschreiben

Boss 1: Die Gildenhalle



DS 12

60 Minuten · ≈ 24 BE
Zeitbudget wie im Abi: ≈ 2,5 min pro BE

Papier, ohne Gerät
Abi-Operatoren und BE-Angaben an jeder Teilaufgabe

Neuer Kontext
kein Held, kein Monster – gleiche Konzepte, Transfer

Zusätzlich im Lab
Online-Boss „Der Reitstall der Heldenschmiede“ · 25 min

Vorbereitung: Lab-Level 1.1–1.4 · Warm-up-Trainer · Abi-Check · Klassenkarten-Regeln · Kernpunkte „erläutern“